

Capítulo 2: Trabalhando com dados de texto

Este capítulo cobre:

- Preparação de texto para o treinamento de modelos de linguagem de grande porte (LLMs).
- Divisão de texto em *tokens* de palavras e subpalavras.
- Codificação por pares de bytes (*Byte Pair Encoding* - BPE) como uma forma mais avançada de tokenizar texto.
- Amostragem de exemplos de treinamento com uma abordagem de janela deslizante.
- Conversão de tokens em vetores para alimentar um modelo de linguagem de grande porte.

Até agora, cobrimos a estrutura geral dos modelos de linguagem de grande porte (LLMs) e aprendemos que eles são pré-treinados em grandes volumes de texto. Especificamente, nosso foco foi nos LLMs do tipo *decoder-only* (apenas decodificador) baseados na arquitetura transformer, que fundamentam os modelos usados no ChatGPT e em outros LLMs populares no estilo GPT.

Durante a etapa de pré-treinamento, os LLMs processam o texto uma palavra por vez. Treinar LLMs com milhões a bilhões de parâmetros usando uma tarefa de previsão da próxima palavra resulta em modelos com capacidades impressionantes. Esses modelos podem, então, ser ajustados (*fine-tuning*) para seguir instruções gerais ou executar tarefas-alvo específicas. Mas antes de podermos implementar e treinar LLMs, precisamos preparar o conjunto de dados de treinamento, conforme ilustrado na Figura 2.1.

Figura 2.1

As três etapas principais da codificação de um LLM. Este capítulo foca no passo 1 da Etapa 1: implementação do pipeline de amostragem de dados.

Construindo um LLM:

- **Etapa 1:**
 1. **Preparação de dados & amostragem** (*Implementa o pipeline de amostragem de dados*)
 2. Mecanismo de atenção
 3. Arquitetura do LLM
- **Etapa 2:**
 - 4) Pré-treinamento
 - 5) Loop de treinamento
 - 6) Avaliação do modelo
 - 7) Carregamento de pesos pré-treinados

(*Pré-treina o LLM em dados não rotulados para obter um modelo base para posterior ajuste fino*)
- **Etapa 3:**
 - 8) **Ajuste fino (Fine-tuning)** → *Conjunto de dados com rótulos de classe* →

Classificador (Ajusta o LLM pré-treinado para criar um modelo de classificação)
9) **Ajuste fino (Fine-tuning)** → *Conjunto de dados de instruções* → *Assistente pessoal (Ajusta o LLM pré-treinado para criar um assistente pessoal ou modelo de chat)*

Você aprenderá como preparar o texto de entrada para o treinamento de LLMs. Isso envolve dividir o texto em tokens individuais de palavras e subpalavras, que podem então ser codificados em representações vetoriais para o LLM. Você também aprenderá sobre esquemas avançados de tokenização, como a codificação por pares de bytes (*Byte Pair Encoding* - BPE), que é utilizada em LLMs populares como o GPT. Por fim, implementaremos uma estratégia de amostragem e carregamento de dados para produzir os pares de entrada-saída necessários para o treinamento de LLMs.

2.1 Compreendendo embeddings de palavras

Modelos de redes neurais profundas, incluindo LLMs, não podem processar texto bruto diretamente. Como o texto é categórico, ele não é compatível com as operações matemáticas usadas para implementar e treinar redes neurais. Portanto, precisamos de uma maneira de representar palavras como vetores de valores contínuos.

NOTA: Leitores não familiarizados com vetores e tensores em um contexto computacional podem aprender mais no Apêndice A, seção A.2.2.

O conceito de converter dados em um formato vetorial é frequentemente chamado de **embedding** (incorporação). Usando uma camada específica de rede neural ou outro modelo de rede neural pré-treinado, podemos embutir diferentes tipos de dados — por exemplo, vídeo, áudio e texto, conforme ilustrado na Figura 2.2. No entanto, é importante notar que diferentes formatos de dados exigem modelos de embedding distintos. Por exemplo, um modelo de embedding projetado para texto não seria adequado para embutir dados de áudio ou vídeo.

Figura 2.2

Modelos de aprendizado profundo não podem processar formatos de dados como vídeo, áudio e texto em sua forma bruta. Assim, usamos um modelo de embedding para transformar esses dados brutos em uma representação vetorial densa que as arquiteturas de aprendizado profundo possam entender e processar facilmente. Especificamente, esta figura ilustra o processo de conversão de dados brutos em um vetor numérico tridimensional.

- **Amostra de vídeo** → *Modelo de embedding de vídeo* → **Vetor de embedding de vídeo:** [1.23, -0.31, 0.89]
- **Amostra de áudio** → *Modelo de embedding de áudio* → **Vetor de embedding de áudio:** [-0.15, 0.45, 2.11]
- **Amostra de texto** → *Modelo de embedding de texto* → **Vetor de embedding de texto:** [1.78, 0.18, -2.10]

Em sua essência, um embedding é um mapeamento de objetos discretos, como palavras, imagens ou até mesmo documentos inteiros, para pontos em um espaço vetorial contínuo

— o objetivo principal dos embeddings é converter dados não numéricos em um formato que as redes neurais possam processar.

Embora os embeddings de palavras sejam a forma mais comum de embedding de texto, também existem embeddings para frases, parágrafos ou documentos inteiros. Embeddings de frases ou parágrafos são escolhas populares para a geração aumentada por recuperação (*Retrieval-Augmented Generation* - RAG). A geração aumentada por recuperação combina a geração (como a produção de texto) com a recuperação (como a busca em uma base de conhecimento externa) para extrair informações relevantes ao gerar texto, que é uma técnica além do escopo deste livro. Como nosso objetivo é treinar LLMs no estilo GPT, que aprendem a gerar texto uma palavra por vez, focaremos nos embeddings de palavras.

Vários algoritmos e frameworks foram desenvolvidos para gerar embeddings de palavras. Um dos exemplos mais antigos e populares é a abordagem **Word2Vec**. O Word2Vec treinava arquiteturas de redes neurais para gerar embeddings de palavras prevendo o contexto de uma palavra dada a palavra-alvo ou vice-versa. A ideia principal por trás do Word2Vec é que palavras que aparecem em contextos semelhantes tendem a ter significados semelhantes. Consequentemente, quando projetadas em embeddings de palavras bidimensionais para fins de visualização, termos semelhantes se agrupam, como mostrado na Figura 2.3.

Os embeddings de palavras podem ter dimensões variáveis, de uma a milhares. Uma dimensionalidade mais alta pode capturar relações mais sutis, mas ao custo da eficiência computacional.

Figura 2.3

Se os embeddings de palavras forem bidimensionais, podemos plotá-los em um gráfico de dispersão bidimensional para fins de visualização, como mostrado aqui. Ao usar técnicas de embedding de palavras, como Word2Vec, palavras correspondentes a conceitos semelhantes geralmente aparecem próximas umas das outras no espaço de embedding. Por exemplo, diferentes tipos de pássaros aparecem mais próximos uns dos outros no espaço de embedding do que em países e cidades.

- **Dimensão 1 vs. Dimensão 2:**
 - Grupo de aves: *duck* (pato), *goose* (ganso), *eagle* (águia) agrupados em uma região.
 - Animal terrestre: *squirrel* (esquilo) em um vetor diferente.
 - Termos de extensão: *long*, *longer*, *longest*.
 - Países e capitais: *Germany*, *Berlin*, *England*, *London* agrupados com relações análogas.

Embora possamos usar modelos pré-treinados como o Word2Vec para gerar embeddings para modelos de aprendizado de máquina, os LLMs comumente produzem seus próprios embeddings que fazem parte da camada de entrada e são atualizados durante o treinamento. A vantagem de otimizar os embeddings como parte do treinamento do LLM, em vez de usar o Word2Vec, é que os embeddings são otimizados para a tarefa e os dados específicos em questão. Implementaremos essas camadas de embedding mais adiante

neste capítulo. (Os LLMs também podem criar embeddings de saída contextualizados, como discutiremos no Capítulo 3.)

Infelizmente, embeddings de alta dimensão apresentam um desafio para a visualização porque nossa percepção sensorial e as representações gráficas comuns são inerentemente limitadas a três dimensões ou menos, razão pela qual a Figura 2.3 mostra embeddings bidimensionais em um gráfico de dispersão bidimensional. No entanto, ao trabalhar com LLMs, normalmente usamos embeddings com uma dimensionalidade muito maior. Tanto no GPT-2 quanto no GPT-3, o tamanho do embedding (frequentemente chamado de dimensionalidade dos estados ocultos do modelo) varia com base na variante e no tamanho específicos do modelo. É um compromisso entre desempenho e eficiência. Os menores modelos do GPT-2 (117M e 125M de parâmetros) usam um tamanho de embedding de 768 dimensões para fornecer exemplos concretos. O maior modelo do GPT-3 (175B de parâmetros) usa um tamanho de embedding de 12.288 dimensões.

A seguir, passaremos pelas etapas necessárias para preparar os embeddings usados por um LLM, o que inclui dividir o texto em palavras, converter palavras em tokens e transformar tokens em vetores de embedding.

2.2 Tokenizando texto

Vamos discutir como dividimos o texto de entrada em tokens individuais, uma etapa de pré-processamento necessária para criar embeddings para um LLM. Esses tokens são palavras individuais ou caracteres especiais, incluindo caracteres de pontuação, conforme mostrado na Figura 2.4.

Figura 2.4

Uma visão das etapas de processamento de texto no contexto de um LLM. Aqui, dividimos um texto de entrada em tokens individuais, que são palavras ou caracteres especiais, como caracteres de pontuação.

1. **Texto de entrada:** This is an example.
2. **Texto tokenizado:** This | is | an | example | .
3. **IDs de Token:** 40134 | 2052 | 133 | 389 | 12
4. **Embeddings de token:** (Vetores passados para o transformer decoder-only no estilo GPT)
5. **Etapas de pós-processamento** → **Texto de saída**

O texto que iremos tokenizar para o treinamento do LLM é "The Verdict" (O Veredito), um conto de Edith Wharton, que entrou em domínio público e, portanto, pode ser usado para tarefas de treinamento de LLMs. O texto está disponível no Wikisource em [\[https://en.wikisource.org/wiki/The_Verdict\]](https://en.wikisource.org/wiki/The_Verdict)(https://en.wikisource.org/wiki/The_Verdict).

Alternativamente, você pode encontrar o arquivo [the-verdict.txt](https://mng.bz/Adng) no repositório do GitHub deste livro em [\[https://mng.bz/Adng\]](https://mng.bz/Adng)(<https://mng.bz/Adng>). Você pode baixar o arquivo com o seguinte código Python:

```
Python
import urllib.request
```

```
url = (  
    "https://raw.githubusercontent.com/rasbt/"  
    "LLMs-from-scratch/main/ch02/01_main-chapter-code/"  
    "the-verdict.txt"  
)  
file_path = "the-verdict.txt"  
urllib.request.urlretrieve(url, file_path)
```

Em seguida, podemos carregar o arquivo `the-verdict.txt` usando os utilitários padrão de leitura de arquivos do Python.

Listagem 2.1: Lendo um conto como amostra de texto no Python

```
Python  
with open("the-verdict.txt", "r", encoding="utf-8") as f:  
    raw_text = f.read()  
  
print("Total number of character:", len(raw_text))  
print(raw_text[:99])
```

O comando `print` exibe o número total de caracteres seguido pelos primeiros 100 caracteres deste arquivo para fins de ilustração:

```
Plaintext  
Total number of character: 20479  
I HAD always thought Jack Gisburn rather a cheap genius--though a good fellow enough--so  
it was no
```

Nosso objetivo é tokenizar este conto de 20.479 caracteres em palavras individuais e caracteres especiais que possamos, em seguida, transformar em embeddings para o treinamento do LLM.

NOTA: É comum processar milhões de artigos e centenas de milhares de livros — muitos gigabytes de texto — ao trabalhar com LLMs. No entanto, para fins educacionais, é suficiente trabalhar com amostras de texto menores, como um único livro, para ilustrar as ideias principais por trás das etapas de processamento de texto e tornar possível executá-lo em um tempo razoável em hardware de consumo.

Como podemos dividir melhor este texto para obter uma lista de tokens? Para isso, faremos uma pequena excursão e usaremos a biblioteca de expressões regulares do Python, `re`, para fins de ilustração. (Você não precisa aprender ou memorizar nenhuma sintaxe de expressão regular, pois mais tarde mudaremos para um tokenizador pré-construído.)

Usando um texto de exemplo simples, podemos usar o comando `re.split` com a seguinte sintaxe para dividir um texto em caracteres de espaço em branco:

```
Python
```

```
import re

text = "Hello, world. This, is a test."
result = re.split(r'(\s)', text)
print(result)
```

O resultado é uma lista de palavras individuais, espaços em branco e caracteres de pontuação:

```
Python
['Hello,', ' ', 'world.', ' ', 'This,', ' ', 'is', ' ', 'a', ' ', 'test.']
```

Esse esquema simples de tokenização funciona em sua maior parte para separar o texto de exemplo em palavras individuais; no entanto, algumas palavras ainda estão conectadas a caracteres de pontuação que queremos ter como entradas de lista separadas. Também evitamos deixar todo o texto em minúsculas porque a capitalização ajuda os LLMs a distinguir entre substantivos próprios e comuns, entender a estrutura da frase e aprender a gerar texto com a capitalização correta.

Vamos modificar a expressão regular para dividir em espaços em branco (**s**), vírgulas e pontos (**[,.]**):

```
Python
result = re.split(r'([,.]|\s)', text)
print(result)
```

Podemos ver que as palavras e os caracteres de pontuação agora são entradas de lista separadas, exatamente como queríamos:

```
Python
['Hello', ',', ' ', 'world', '.', ' ', 'This', ',', ' ', 'is', ' ', 'a', ' ', 'test', '.', ' ']
```

Um pequeno problema restante é que a lista ainda inclui caracteres de espaço em branco. Opcionalmente, podemos remover esses caracteres redundantes com segurança da seguinte forma:

```
Python
result = [item for item in result if item.strip()]
print(result)
```

A saída resultante sem espaços em branco fica assim:

```
Python
['Hello', ',', 'world', '.', 'This', ',', 'is', 'a', 'test', '.']
```

NOTA: Ao desenvolver um tokenizador simples, se devemos codificar espaços em branco como caracteres separados ou apenas removê-los depende da nossa aplicação e de seus requisitos. Remover espaços em branco reduz a memória e os requisitos computacionais. No entanto, manter espaços em

branco pode ser útil se treinarmos modelos sensíveis à estrutura exata do texto (por exemplo, código Python, que é sensível à indentação e ao espaçamento). Aqui, removemos espaços em branco por simplicidade e brevidade das saídas tokenizadas. Mais tarde, mudaremos para um esquema de tokenização que inclui espaços em branco.

O esquema de tokenização que concebemos aqui funciona bem no texto de amostra simples. Vamos modificá-lo um pouco mais para que ele também possa lidar com outros tipos de pontuação, como pontos de interrogação, aspas e os travessões duplos que vimos anteriormente nos primeiros 100 caracteres do conto de Edith Wharton, junto com caracteres especiais adicionais:

```
Python
text = "Hello, world. Is this a test?"
result = re.split(r'([.,:;?_!"()\[\]{}--|\s])', text)
result = [item.strip() for item in result if item.strip()]
print(result)
```

A saída resultante é:

```
Python
['Hello', ',', 'world', '.', 'Is', 'this', 'a', 'test', '?']
```

Como podemos ver com base nos resultados resumidos na Figura 2.5, nosso esquema de tokenização agora pode lidar com sucesso com os vários caracteres especiais no texto.

Figura 2.5

O esquema de tokenização que implementamos até agora divide o texto em palavras individuais e caracteres de pontuação. Neste exemplo específico, o texto de amostra é dividido em 10 tokens individuais.

- **Texto de entrada:** Hello, world. Is this a test?
- **Texto tokenizado:** Hello | , | world | . | Is | this | a | test | ?

Agora que temos um tokenizador básico funcionando, vamos aplicá-lo a todo o conto de Edith Wharton:

```
Python
preprocessed = re.split(r'([.,:;?_!"()\[\]{}--|\s])', raw_text)
preprocessed = [item.strip() for item in preprocessed if item.strip()]
print(len(preprocessed))
```

Esta instrução `print` exibe `4690`, que é o número de tokens neste texto (sem espaços em branco). Vamos imprimir os primeiros 30 tokens para uma verificação visual rápida:

```
Python
print(preprocessed[:30])
```

A saída resultante mostra que nosso tokenizador parece estar lidando bem com o texto, já que todas as palavras e caracteres especiais estão nitidamente separados:

Python

```
['I', 'HAD', 'always', 'thought', 'Jack', 'Gisburn', 'rather', 'a', 'cheap', 'genius', '--', 'though', 'a', 'good', 'fellow', 'enough', '--', 'so', 'it', 'was', 'no', 'great', 'surprise', 'to', 'me', 'to', 'hear', 'that', ',', 'in']
```

2.3 Convertendo tokens em IDs de token

Em seguida, vamos converter esses tokens de uma string Python para uma representação de número inteiro para produzir os IDs de token. Essa conversão é uma etapa intermediária antes de converter os IDs de token em vetores de embedding.

Para mapear os tokens gerados anteriormente em IDs de token, precisamos primeiro construir um vocabulário. Este vocabulário define como mapeamos cada palavra única e caractere especial para um inteiro único, conforme mostrado na Figura 2.6.

Figura 2.6

Construímos um vocabulário tokenizando todo o texto em um conjunto de dados de treinamento em tokens individuais. Esses tokens individuais são então ordenados alfabeticamente e os tokens duplicados são removidos. Os tokens únicos são então agregados em um vocabulário que define um mapeamento de cada token único para um valor inteiro único. O vocabulário retratado é propositalmente pequeno e não contém pontuação ou caracteres especiais por simplicidade.

1. **Conjunto de dados de treinamento completo:** "The quick brown fox jumps over the lazy dog"
2. **Conjunto de dados tokenizado:** The, quick, brown, fox, jumps, over, the, lazy, dog
3. **Vocabulário (Tokens únicos → IDs de Token):**
 - brown: 0
 - dog: 1
 - fox: 2
 - jumps: 3
 - lazy: 4
 - over: 5
 - quick: 6
 - the / The: 7

Agora que tokenizamos o conto de Edith Wharton e o atribuímos a uma variável Python chamada `preprocessed`, vamos criar uma lista de todos os tokens únicos e ordená-los alfabeticamente para determinar o tamanho do vocabulário:

Python

```
all_words = sorted(set(preprocessed))
vocab_size = len(all_words)
print(vocab_size)
```

Após determinar que o tamanho do vocabulário é 1.130 por meio deste código, criamos o vocabulário e imprimimos suas primeiras 51 entradas para fins de ilustração.

Listagem 2.2: Criando um vocabulário

```
Python
vocab = {token: integer for integer, token in enumerate(all_words)}
for i, item in enumerate(vocab.items()):
    print(item)
    if i >= 50:
        break
```

A saída é:

```
Python
('!', 0)
('"'', 1)
('"'', 2)
...
('Her', 49)
('Hermia', 50)
```

Como podemos ver, o dicionário contém tokens individuais associados a rótulos inteiros únicos. Nosso próximo objetivo é aplicar este vocabulário para converter novos textos em IDs de token (Figura 2.7).

Figura 2.7

Começando com uma nova amostra de texto, tokenizamos o texto e usamos o vocabulário para converter os tokens de texto em IDs de token. O vocabulário é construído a partir de todo o conjunto de dados de treinamento e pode ser aplicado ao próprio conjunto de treinamento e a quaisquer novas amostras de texto. O vocabulário retratado não contém pontuação ou caracteres especiais por simplicidade.

- **Texto de amostra:** "The brown dog playfully chased the swift fox"
- **Texto tokenizado:** The | brown | dog
- **Vocabulário existente:** brown: 0, dog: 1, fox: 2... the: 7
- **IDs de token:** 7 | 0 | 1

Quando queremos converter as saídas de um LLM de números de volta em texto, precisamos de uma maneira de transformar IDs de token em texto. Para isso, podemos criar uma versão inversa do vocabulário que mapeia IDs de token de volta para os tokens de texto correspondentes.

Vamos implementar uma classe completa de tokenizador em Python com um método `encode` que divide o texto em tokens e realiza o mapeamento de string para inteiro para produzir IDs de token por meio do vocabulário. Além disso, implementaremos um método `decode` que realiza o mapeamento inverso de inteiro para string para converter os IDs de token de volta em texto. A listagem a seguir mostra o código para esta implementação de tokenizador.

Listagem 2.3: Implementando um tokenizador de texto simples

Python

```
class SimpleTokenizerV1:
    def __init__(self, vocab):
        # Armazena o vocabulário como um atributo de classe para acesso nos métodos
        encode e decode
        self.str_to_int = vocab
        # Cria um vocabulário inverso que mapeia IDs de token de volta aos tokens de texto
        originais
        self.int_to_str = {i: s for s, i in vocab.items()}

    def encode(self, text):
        # Processa o texto de entrada em IDs de token
        preprocessed = re.split(r'([.,:;?!"()\']|--|\s)', text)
        preprocessed = [
            item.strip() for item in preprocessed if item.strip()
        ]
        ids = [self.str_to_int[s] for s in preprocessed]
        return ids

    def decode(self, ids):
        # Converte IDs de token de volta em texto
        text = " ".join([self.int_to_str[i] for i in ids])
        # Remove espaços antes da pontuação especificada
        text = re.sub(r'\s+([.,:;?!"()\']|--|\s)', r'\1', text)
        return text
```

Usando a classe Python `SimpleTokenizerV1`, podemos agora instanciar novos objetos de tokenizador por meio de um vocabulário existente, que podemos usar para codificar e decodificar texto, conforme ilustrado na Figura 2.8.

Figura 2.8

Implementações de tokenizadores compartilham dois métodos comuns: um método `encode` e um método `decode`. O método `encode` recebe o texto de amostra, divide-o em tokens individuais e converte os tokens em IDs de token por meio do vocabulário. O método `decode` recebe IDs de token, converte-os de volta em tokens de texto e concatena os tokens de texto em texto natural.

Vamos instanciar um novo objeto tokenizador da classe `SimpleTokenizerV1` e tokenizar um trecho do conto de Edith Wharton para testá-lo na prática:

Python

```
tokenizer = SimpleTokenizerV1(vocab)
text = """It's the last he painted, you know," Mrs. Gisburn said with pardonable pride."""
ids = tokenizer.encode(text)
print(ids)
```

O código anterior imprime os seguintes IDs de token:

Python

```
[1, 56, 2, 850, 988, 602, 533, 746, 5, 1126, 596, 5, 1, 67, 7, 38, 851, 1108, 754, 793, 7]
```

Em seguida, vamos ver se podemos transformar esses IDs de token de volta em texto usando o método `decode`:

Python

```
print(tokenizer.decode(ids))
```

Isso exibe:

Plaintext

```
"It's the last he painted, you know," Mrs. Gisburn said with pardonable pride.'
```

Com base nessa saída, podemos ver que o método `decode` converteu com sucesso os IDs de token de volta ao texto original.

Até agora, tudo bem. Implementamos um tokenizador capaz de tokenizar e detokenizar texto com base em um trecho do conjunto de treinamento. Vamos agora aplicá-lo a uma nova amostra de texto não contida no conjunto de treinamento:

Python

```
text = "Hello, do you like tea?"  
print(tokenizer.encode(text))
```

A execução deste código resultará no seguinte erro:

Plaintext

```
KeyError: 'Hello'
```

O problema é que a palavra "Hello" não foi usada no conto "The Verdict". Portanto, ela não está contida no vocabulário. Isso destaca a necessidade de considerar conjuntos de dados de treinamento grandes e diversos para expandir o vocabulário ao trabalhar com LLMs.

Em seguida, testaremos o tokenizador mais detalhadamente em textos que contêm palavras desconhecidas e discutiremos tokens de contexto especiais adicionais que podem ser usados para fornecer contexto adicional a um LLM durante o treinamento.

2.4 Adicionando tokens de contexto especiais

Precisamos modificar o tokenizador para lidar com palavras desconhecidas. Também precisamos abordar o uso e a adição de tokens de contexto especiais que podem aprimorar a compreensão do modelo sobre o contexto ou outras informações relevantes no texto. Esses tokens especiais podem incluir marcadores para palavras desconhecidas e limites de documentos, por exemplo. Em particular, modificaremos o vocabulário e o tokenizador, `SimpleTokenizerV2`, para suportar dois novos tokens: `<|junk|>` e `<|endoftext|>`, conforme ilustrado na Figura 2.9.

Figura 2.9

Adicionamos tokens especiais a um vocabulário para lidar com certos contextos. Por exemplo, adicionamos um token `<|unk|>` para representar palavras novas e desconhecidas que não faziam parte dos dados de treinamento e, portanto, não faziam parte do vocabulário existente. Além disso, adicionamos um token `<|endoftext|>` que podemos usar para separar duas fontes de texto não relacionadas.

- **Vocabulário existente + extensão:**
 - `brown`: 0, `dog`: 1, `fox`: 2 ...
 - `<|unk|>`: 783
 - `<|endoftext|>`: 784

Podemos modificar o tokenizador para usar um token `<|unk|>` se ele encontrar uma palavra que não faz parte do vocabulário. Além disso, adicionamos um token entre textos não relacionados. Por exemplo, ao treinar LLMs no estilo GPT em vários documentos ou livros independentes, é comum inserir um token antes de cada documento ou livro que segue uma fonte de texto anterior, conforme ilustrado na Figura 2.10. Isso ajuda o LLM a entender que, embora essas fontes de texto estejam concatenadas para o treinamento, elas são, na verdade, não relacionadas.

Figura 2.10

Ao trabalhar com várias fontes de texto independentes, adicionamos tokens `<|endoftext|>` entre esses textos. Esses tokens `<|endoftext|>` atuam como marcadores, sinalizando o início ou o fim de um segmento específico, permitindo um processamento e compreensão mais eficazes pelo LLM.

Vamos agora modificar o vocabulário para incluir estes dois tokens especiais, `<|unk|>` e `<|endoftext|>`, adicionando-os à nossa lista de todas as palavras únicas:

Python

```
all_tokens = sorted(list(set(preprocessed)))
all_tokens.extend(["<|endoftext|>", "<|unk|>"])
vocab = {token: integer for integer, token in enumerate(all_tokens)}
print(len(vocab.items()))
```

Com base na saída desta instrução `print`, o novo tamanho do vocabulário é 1.132 (o tamanho do vocabulário anterior era 1.130).

Como uma verificação rápida adicional, vamos imprimir as últimas cinco entradas do vocabulário atualizado:

Python

```
for i, item in enumerate(list(vocab.items())[-5:]):
    print(item)
```

O código imprime:

Python

```
('younger', 1127)
('your', 1128)
('yourself', 1129)
('<|endoftext|>', 1130)
('<|unk|>', 1131)
```

Com base na saída do código, podemos confirmar que os dois novos tokens especiais foram de fato incorporados com sucesso ao vocabulário. Em seguida, ajustamos o tokenizador da listagem de código 2.3 de acordo, como mostrado na listagem a seguir.

Listagem 2.4: Um tokenizador de texto simples que lida com palavras desconhecidas

Python

```
class SimpleTokenizerV2:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i: s for s, i in vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([.,:;?_!"\']|--|\s)', text)
        preprocessed = [
            item.strip() for item in preprocessed if item.strip()
        ]
        # Substitui palavras desconhecidas por tokens <|unk|>
        preprocessed = [
            item if item in self.str_to_int else "<|unk|>"
            for item in preprocessed
        ]
        ids = [self.str_to_int[s] for s in preprocessed]
        return ids

    def decode(self, ids):
        text = " ".join([self.int_to_str[i] for i in ids])
        # Remove espaços antes das pontuações especificadas
        text = re.sub(r'\s+([.,:;?_!"\']|--|\s)', r'\1', text)
        return text
```

Em comparação com o `SimpleTokenizerV1` que implementamos na listagem 2.3, o novo `SimpleTokenizerV2` substitui palavras desconhecidas por tokens `<|unk|>`.

Vamos agora experimentar este novo tokenizador na prática. Para isso, usaremos uma amostra de texto simples que concatenamos a partir de duas frases independentes e não relacionadas:

Python

```
text1 = "Hello, do you like tea?"
text2 = "In the sunlit terraces of the palace."
```

```
text = " <|endoftext|> ".join((text1, text2))
print(text)
```

A saída é:

Plaintext

Hello, do you like tea? <|endoftext|> In the sunlit terraces of the palace.

Em seguida, vamos tokenizar o texto de amostra usando o `SimpleTokenizerV2` no `vocab` que criamos anteriormente na listagem 2.2:

Python

```
tokenizer = SimpleTokenizerV2(vocab)
print(tokenizer.encode(text))
```

Isso imprime os seguintes IDs de token:

Python

```
[1131, 5, 355, 1126, 628, 975, 10, 1130, 55, 988, 956, 984, 722, 988, 1131, 7]
```

Podemos ver que a lista de IDs de token contém `1130` para o token separador `<|endoftext|>` bem como dois tokens `1131`, que são usados para palavras desconhecidas.

Vamos detokenizar o texto para uma verificação rápida de sanidade:

Python

```
print(tokenizer.decode(tokenizer.encode(text)))
```

A saída é:

Plaintext

<|unk|>, do you like tea? <|endoftext|> In the sunlit terraces of the <|unk|>.

Com base na comparação deste texto detokenizado com o texto de entrada original, sabemos que o conjunto de dados de treinamento, o conto "The Verdict" de Edith Wharton, não contém as palavras "Hello" e "palace".

Dependendo do LLM, alguns pesquisadores também consideram tokens especiais adicionais, tais como:

- **[BOS] (*beginning of sequence* - início da sequência):** Este token marca o início de um texto. Ele sinaliza ao LLM onde um pedaço de conteúdo começa.
- **[EOS] (*end of sequence* - fim da sequência):** Este token é posicionado ao final de um texto e é especialmente útil ao concatenar vários textos não relacionados, semelhante ao `<|endoftext|>`. Por exemplo, ao combinar dois artigos da Wikipédia ou livros diferentes, o token `[EOS]` indica onde um termina e o próximo começa.
- **[PAD] (*padding* - preenchimento):** Ao treinar LLMs com tamanhos de lote (*batch sizes*) maiores que um, o lote pode conter textos de comprimentos variados. Para garantir que todos os textos tenham o mesmo comprimento, os textos mais curtos

são estendidos ou "preenchidos" usando o token `[PAD]`, até o comprimento do texto mais longo no lote.

O tokenizador usado para modelos GPT não precisa de nenhum desses tokens; ele usa apenas um token `<|endoftext|>` por simplicidade. `<|endoftext|>` é análogo ao token `[EOS]`. `<|endoftext|>` também é usado para preenchimento. No entanto, como exploraremos em capítulos subsequentes, ao treinar com entradas em lote, normalmente usamos uma máscara, o que significa que não prestamos atenção aos tokens preenchidos. Assim, o token específico escolhido para preenchimento torna-se inconsequente.

Além disso, o tokenizador usado para modelos GPT também não usa um token `<|unk|>` para palavras fora do vocabulário. Em vez disso, os modelos GPT usam um tokenizador de codificação por pares de bytes (*Byte Pair Encoding*), que divide as palavras em unidades de subpalavras, o que discutiremos a seguir.

2.5 Codificação por pares de bytes (Byte Pair Encoding - BPE)

Vamos examinar um esquema de tokenização mais sofisticado com base em um conceito chamado **Byte Pair Encoding (BPE)**. O tokenizador BPE foi usado para treinar LLMs como GPT-2, GPT-3 e o modelo original usado no ChatGPT.

Como implementar o BPE pode ser relativamente complicado, usaremos uma biblioteca código aberto Python existente chamada `tiktoken` ([\[https://github.com/openai/tiktoken\]](https://github.com/openai/tiktoken)), que implementa o algoritmo BPE de forma muito eficiente com base em código-fonte em Rust. Semelhante a outras bibliotecas Python, podemos instalar a biblioteca `tiktoken` via instalador `pip` do Python a partir do terminal:

```
Bash
pip install tiktoken
```

O código que usaremos é baseado no `tiktoken 0.7.0`. Você pode usar o seguinte código para verificar a versão que possui instalada atualmente:

```
Python
from importlib.metadata import version
import tiktoken

print("tiktoken version:", version("tiktoken"))
```

Uma vez instalado, podemos instanciar o tokenizador BPE do `tiktoken` da seguinte forma:

```
Python
tokenizer = tiktoken.get_encoding("gpt2")
```

O uso deste tokenizador é semelhante ao `SimpleTokenizerV2` que implementamos anteriormente por meio de um método `encode`:

```

Python
text = (
    "Hello, do you like tea? <|endoftext|> In the sunlit terraces"
    "of someunknownPlace."
)
integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
print(integers)

```

O código imprime os seguintes IDs de token:

```

Python
[15496, 11, 466, 345, 588, 8887, 30, 220, 50256, 554, 262, 4252, 18250, 8812, 2114, 286,
617, 34680, 27271, 13]

```

Podemos então converter os IDs de token de volta em texto usando o método `decode`, semelhante ao nosso `SimpleTokenizerV2`:

```

Python
strings = tokenizer.decode(integers)
print(strings)

```

O código imprime:

```

Plaintext
Hello, do you like tea? <|endoftext|> In the sunlit terraces of someunknownPlace.

```

Podemos fazer duas observações notáveis com base nos IDs de token e no texto decodificado.

Primeiro, o token `<|endoftext|>` é atribuído a um ID de token relativamente grande, a saber, **50256**. De fato, o tokenizador BPE, que foi usado para treinar modelos como GPT-2, GPT-3 e o modelo original usado no ChatGPT, tem um tamanho de vocabulário total de 50.257, com o `<|endoftext|>` sendo atribuído ao maior ID de token.

Segundo, o tokenizador BPE codifica e decodifica palavras desconhecidas, como `someunknownPlace`, corretamente. O tokenizador BPE pode lidar com qualquer palavra desconhecida. Como ele consegue isso sem usar tokens `<|junk|>`?

O algoritmo subjacente ao BPE descompõe palavras que não estão em seu vocabulário pré-definido em unidades de subpalavras menores ou até mesmo em caracteres individuais, permitindo que ele lide com palavras fora do vocabulário. Assim, graças ao algoritmo BPE, se o tokenizador encontrar uma palavra desconhecida durante a tokenização, ele pode representá-la como uma sequência de tokens de subpalavras ou caracteres, conforme ilustrado na Figura 2.11.

Figura 2.11

Tokenizadores BPE descompõem palavras desconhecidas em subpalavras e caracteres individuais. Dessa forma, um tokenizador BPE pode analisar qualquer palavra e não precisa substituir palavras desconhecidas por tokens especiais, como <|unk|>.

- **Amostra de texto com palavras desconhecidas:** "Akwiw ier"
- **Descomposição em tokens:** "Ak" | "w" | "ir" | "w" | " " | "ier"
- **IDs de token:** 33901 | 86 | 343 | 86 | 220 | 959

A capacidade de decompor palavras desconhecidas em caracteres individuais garante que o tokenizador e, conseqüentemente, o LLM treinado com ele possam processar qualquer texto, mesmo que ele contenha palavras que não estavam presentes em seus dados de treinamento.

Exercício 2.1: Codificação por pares de bytes de palavras desconhecidas

Experimente o tokenizador BPE da biblioteca `tiktoken` nas palavras desconhecidas "Akwiw ier" e imprima os IDs de token individuais. Em seguida, chame a função `decode` em cada um dos inteiros resultantes nesta lista para reproduzir o mapeamento mostrado na Figura 2.11. Por fim, chame o método `decode` nos IDs de token para verificar se ele pode reconstruir a entrada original, "Akwiw ier".

Uma discussão detalhada e implementação do BPE está fora do escopo deste livro, mas em resumo, ele constrói seu vocabulário mesclando iterativamente caracteres frequentes em subpalavras e subpalavras frequentes em palavras. Por exemplo, o BPE começa adicionando todos os caracteres individuais únicos ao seu vocabulário ("a", "b", etc.). Na etapa seguinte, ele mescla combinações de caracteres que ocorrem juntas com frequência em subpalavras. Por exemplo, "d" e "e" podem ser mesclados na subpalavra "de", que é comum em muitas palavras em inglês como "define", "depend", "made" e "hidden". As fusões são determinadas por um limite de frequência.

2.6 Amostragem de dados com uma janela deslizante

O próximo passo na criação de embeddings para o LLM é gerar os pares de entrada-alvo necessários para o treinamento de um LLM. Como são esses pares de entrada-alvo? Como já aprendemos, os LLMs são pré-treinados prevendo a próxima palavra em um texto, conforme retratado na Figura 2.12.

Figura 2.12

Dada uma amostra de texto, extraia blocos de entrada como subamostras que servem como entrada para o LLM, e a tarefa de previsão do LLM durante o treinamento é prever a próxima palavra que segue o bloco de entrada. Durante o treinamento, mascaramos todas as palavras que estão além do alvo. Observe que o texto mostrado nesta figura deve passar por tokenização antes que o LLM possa processá-lo; no entanto, esta figura omite a etapa de tokenização para fins de clareza.

- **Amostra de texto:** "LLMs learn to predict one word at a time"
- **Entrada recebida pelo LLM → Alvo a prever:**
 - LLMs → learn

- LLMs learn → to
- LLMs learn to → predict
- LLMs learn to predict → one
- LLMs learn to predict one → word
- LLMs learn to predict one word → at
- LLMs learn to predict one word at → a
- LLMs learn to predict one word at a → time

Vamos implementar um carregador de dados (*data loader*) que busca os pares de entrada-alvo da Figura 2.12 a partir do conjunto de dados de treinamento usando uma abordagem de janela deslizante. Para começar, tokenizaremos todo o conto "The Verdict" usando o tokenizador BPE:

Python

```
with open("the-verdict.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()
```

```
enc_text = tokenizer.encode(raw_text)
print(len(enc_text))
```

A execução deste código retornará **5145**, o número total de tokens no conjunto de treinamento, após a aplicação do tokenizador BPE.

Em seguida, removemos os primeiros 50 tokens do conjunto de dados para fins de demonstração, pois isso resulta em um trecho de texto um pouco mais interessante nos passos seguintes:

Python

```
enc_sample = enc_text[50:]
```

Uma das maneiras mais fáceis e intuitivas de criar os pares de entrada-alvo para a tarefa de previsão da próxima palavra é criar duas variáveis, **x** e **y**, onde **x** contém os tokens de entrada e **y** contém os alvos, que são as entradas deslocadas por 1:

Python

```
context_size = 4
# O tamanho do contexto determina quantos tokens estão incluídos na entrada
x = enc_sample[:context_size]
y = enc_sample[1:context_size+1]
print(f"x: {x}")
print(f"y: {y}")
```

Executar o código anterior imprime a seguinte saída:

Plaintext

```
x: [290, 4920, 2241, 287]
y: [4920, 2241, 287, 257]
```

Ao processar as entradas junto com os alvos, que são as entradas deslocadas em uma posição, podemos criar as tarefas de previsão da próxima palavra (veja a Figura 2.12), como segue:

```
Python
for i in range(1, context_size+1):
    context = enc_sample[:i]
    desired = enc_sample[i]
    print(context, "---->", desired)
```

O código imprime:

```
Plaintext
[290] ----> 4920
[290, 4920] ----> 2241
[290, 4920, 2241] ----> 287
[290, 4920, 2241, 287] ----> 257
```

Tudo à esquerda da seta (---->) refere-se à entrada que um LLM receberia, e o ID de token no lado direito da seta representa o ID de token alvo que o LLM deve prever. Vamos repetir o código anterior, mas convertendo os IDs de token em texto:

```
Python
for i in range(1, context_size+1):
    context = enc_sample[:i]
    desired = enc_sample[i]
    print(tokenizer.decode(context), "---->", tokenizer.decode([desired]))
```

As seguintes saídas mostram como a entrada e as saídas se parecem no formato de texto:

```
Plaintext
and ----> established
and established ----> himself
and established himself ----> in
and established himself in ----> a
```

Agora criamos os pares de entrada-alvo que podemos usar para o treinamento do LLM.

Resta apenas mais uma tarefa antes de podermos transformar os tokens em embeddings: implementar um carregador de dados eficiente que itere sobre o conjunto de dados de entrada e retorne as entradas e alvos como tensores do PyTorch, que podem ser pensados como arrays multidimensionais. Em particular, estamos interessados em retornar dois tensores: um tensor de entrada contendo o texto que o LLM vê e um tensor alvo que inclui os alvos para o LLM prever, conforme retratado na Figura 2.13. Embora a figura mostre os tokens no formato de string para fins de ilustração, a implementação do código operará diretamente nos IDs de token, uma vez que o método `encode` do tokenizador BPE realiza tanto a tokenização quanto a conversão em IDs de token em uma única etapa.

Figura 2.13

Para implementar carregadores de dados eficientes, coletamos as entradas em um tensor, x , onde cada linha representa um contexto de entrada. Um segundo tensor, y , contém os alvos de previsão correspondentes (próximas palavras), que são criados deslocando a entrada em uma posição.

- **Texto de amostra:** *"In the heart of the city stood the old library, a relic from a bygone era. Its stone walls bore the marks of time, and ivy clung tightly to its facade..."*
- **Tensor de entradas (\$x\$):**
 - Linha 1: ["In", "the", "heart", "of"]
 - Linha 2: ["the", "city", "stood", "the"]
 - Linha 3: ["old", "library", ",", "a"]
- **Tensor de alvos (\$y\$):**
 - Linha 1: ["the", "heart", "of", "the"]
 - Linha 2: ["city", "stood", "the", "old"]
 - Linha 3: ["library", ",", "a", "relic"]

NOTA: Para a implementação eficiente do carregador de dados, usaremos as classes nativas `Dataset` e `DataLoader` do PyTorch. Para informações adicionais e orientação sobre como instalar o PyTorch, consulte a seção A.2.1.3 no Apêndice A.

O código para a classe de conjunto de dados é mostrado na listagem a seguir.

Listagem 2.5: Um conjunto de dados para entradas e alvos em lote

Python

```
import torch
```

```
from torch.utils.data import Dataset, DataLoader
```

```
class GPTDatasetV1(Dataset):
```

```
    def __init__(self, txt, tokenizer, max_length, stride):
```

```
        self.input_ids = []
```

```
        self.target_ids = []
```

```
        # Tokeniza todo o texto
```

```
        token_ids = tokenizer.encode(txt)
```

```
        # Usa uma janela deslizante para dividir o livro em sequências sobrepostas de  
        max_length
```

```
        for i in range(0, len(token_ids) - max_length, stride):
```

```
            input_chunk = token_ids[i:i + max_length]
```

```
            target_chunk = token_ids[i + 1: i + max_length + 1]
```

```
            self.input_ids.append(torch.tensor(input_chunk))
```

```
            self.target_ids.append(torch.tensor(target_chunk))
```

```
    def __len__(self):
```

```
        # Retorna o número total de linhas no conjunto de dados
```

```

return len(self.input_ids)

def __getitem__(self, idx):
    # Retorna uma única linha do conjunto de dados
    return self.input_ids[idx], self.target_ids[idx]

```

A classe `GPTDatasetV1` é baseada na classe `Dataset` do PyTorch e define como linhas individuais são buscadas do conjunto de dados, onde cada linha consiste em um número de IDs de token (com base em um `max_length`) atribuídos a um tensor `input_chunk`. O tensor `target_chunk` contém os alvos correspondentes. Recomendo continuar lendo para ver como são os dados retornados deste conjunto de dados quando combinamos o conjunto de dados com um `DataLoader` do PyTorch — isso trará intuição e clareza adicionais.

NOTA: Se você é novo na estrutura das classes `Dataset` do PyTorch, como mostrado na listagem 2.5, consulte a seção A.6 no Apêndice A, que explica a estrutura geral e o uso das classes `Dataset` e `DataLoader` do PyTorch.

O código a seguir usa o `GPTDatasetV1` para carregar as entradas em lotes por meio de um `DataLoader` do PyTorch.

Listagem 2.6: Um carregador de dados para gerar lotes com pares de entrada-alvo

Python

```

def create_dataloader_v1(txt, batch_size=4, max_length=256,
                        stride=128, shuffle=True, drop_last=True,
                        num_workers=0):
    # Inicializa o tokenizador
    tokenizer = tiktoken.get_encoding("gpt2")

    # Cria o conjunto de dados
    dataset = GPTDatasetV1(txt, tokenizer, max_length, stride)

    dataloader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        # drop_last=True descarta o último lote se for menor que o batch_size especificado
        # para evitar picos de perda durante o treinamento
        drop_last=drop_last,
        # O número de processos de CPU a serem usados para pré-processamento
        num_workers=num_workers
    )
    return dataloader

```

Vamos testar o carregador de dados com um tamanho de lote (*batch size*) de 1 para um LLM com tamanho de contexto de 4 para desenvolver a intuição de como a classe

`GPTDatasetV1` da listagem 2.5 e a função `create_dataloader_v1` da listagem 2.6 funcionam juntas:

Python

```
with open("the-verdict.txt", "r", encoding="utf-8") as f:
```

```
    raw_text = f.read()
```

```
dataloader = create_dataloader_v1(
    raw_text, batch_size=1, max_length=4, stride=1, shuffle=False
)
```

```
data_iter = iter(dataloader)
```

```
# Converte o dataloader em um iterador Python para buscar a próxima entrada por meio da
função nativa next() do Python
```

```
first_batch = next(data_iter)
```

```
print(first_batch)
```

A execução do código anterior imprime o seguinte:

Python

```
[tensor([[ 40, 367, 2885, 1464]]), tensor([[ 367, 2885, 1464, 1807]])]
```

A variável `first_batch` contém dois tensores: o primeiro tensor armazena os IDs de token de entrada e o segundo tensor armazena os IDs de token alvo. Como o `max_length` está definido como 4, cada um dos dois tensores contém quatro IDs de token. Observe que um tamanho de entrada de 4 é bastante pequeno e escolhido apenas por simplicidade. É comum treinar LLMs com tamanhos de entrada de pelo menos 256.

Para entender o significado de `stride=1`, vamos buscar outro lote deste conjunto de dados:

Python

```
second_batch = next(data_iter)
```

```
print(second_batch)
```

O segundo lote tem o seguinte conteúdo:

Python

```
[tensor([[ 367, 2885, 1464, 1807]]), tensor([[2885, 1464, 1807, 3619]])]
```

Se compararmos o primeiro e o segundo lote, podemos ver que os IDs de token do segundo lote estão deslocados em uma posição (por exemplo, o segundo ID na entrada do primeiro lote é 367, que é o primeiro ID da entrada do segundo lote). A configuração `stride` dita o número de posições que as entradas se deslocam entre os lotes, emulando uma abordagem de janela deslizante, como demonstrado na Figura 2.14.

Figura 2.14

Ao criar vários lotes a partir do conjunto de dados de entrada, deslizamos uma janela de entrada pelo texto. Se o passo (stride) for definido como 1, deslocamos a janela de entrada

em uma posição ao criar o próximo lote. Se definirmos o passo igual ao tamanho da janela de entrada, podemos evitar sobreposições entre os lotes.

Exercício 2.2: Carregadores de dados com diferentes passos e tamanhos de contexto

Para desenvolver mais intuição sobre como o carregador de dados funciona, tente executá-lo com diferentes configurações, como `max_length=2` e `stride=2`, e `max_length=8` e `stride=2`.

Tamanhos de lote de 1, como os que amostramos do carregador de dados até agora, são úteis para fins de ilustração. Se você tem experiência prévia com aprendizado profundo, deve saber que tamanhos de lote pequenos exigem menos memória durante o treinamento, mas levam a atualizações de modelo mais ruidosas. Assim como no aprendizado profundo regular, o tamanho do lote é um compromisso e um hiperparâmetro para experimentar ao treinar LLMs.

Vamos examinar rapidamente como podemos usar o carregador de dados para amostrar com um tamanho de lote maior que 1:

Python

```
dataloader = create_dataloader_v1(
    raw_text, batch_size=8, max_length=4, stride=4, shuffle=False
)
data_iter = iter(dataloader)
inputs, targets = next(data_iter)
print("Inputs:\n", inputs)
print("\nTargets:\n", targets)
```

Isso imprime:

Plaintext

Inputs:

```
tensor([[ 40, 367, 2885, 1464],
        [1807, 3619, 402, 271],
        [10899, 2138, 257, 7026],
        [15632, 438, 2016, 257],
        [ 922, 5891, 1576, 438],
        [ 568, 340, 373, 645],
        [1049, 5975, 284, 502],
        [ 284, 3285, 326, 11]])
```

Targets:

```
tensor([[ 367, 2885, 1464, 1807],
        [ 3619, 402, 271, 10899],
        [ 2138, 257, 7026, 15632],
        [ 438, 2016, 257, 922],
        [ 5891, 1576, 438, 568],
        [ 340, 373, 645, 1049],
        [ 5975, 284, 502, 284],
        [ 3285, 326, 11, 367]])
```

```
[ 5975, 284, 502, 284],  
[ 3285, 326, 11, 287]])
```

Observe que aumentamos o passo (*stride*) para 4 para utilizar totalmente o conjunto de dados (não pulamos uma única palavra). Isso evita qualquer sobreposição entre os lotes, pois mais sobreposição poderia levar a um aumento do *overfitting*.

2.7 Criando embeddings de token

A última etapa na preparação do texto de entrada para o treinamento do LLM é converter os IDs de token em vetores de embedding, conforme mostrado na Figura 2.15. Como etapa preliminar, precisamos inicializar esses pesos de embedding com valores aleatórios. Essa inicialização serve como ponto de partida para o processo de aprendizado do LLM. No Capítulo 5, otimizaremos os pesos de embedding como parte do treinamento do LLM.

Figura 2.15

A preparação envolve tokenizar o texto, converter tokens de texto em IDs de token e converter IDs de token em vetores de embedding. Aqui, consideramos os IDs de token criados anteriormente para criar os vetores de embedding de token.

- **Texto de entrada:** This is an example.
- **Texto tokenizado:** This | is | an | example
- **IDs de token:** 40134 | 2052 | 133 | 389 | 12
- **Criando embeddings de token de entrada** → Vetores passados para o transformer decoder-only no estilo GPT

Uma representação vetorial contínua, ou embedding, é necessária uma vez que LLMs no estilo GPT são redes neurais profundas treinadas com o algoritmo de retropropagação (*backpropagation*).

NOTA: Se você não está familiarizado com a forma como as redes neurais são treinadas com retropropagação, leia a seção B.4 no Apêndice A.

Vamos ver como funciona a conversão de ID de token para vetor de embedding com um exemplo prático. Suponha que tenhamos os seguintes quatro tokens de entrada com IDs 2, 3, 5 e 1:

```
Python  
input_ids = torch.tensor([2, 3, 5, 1])
```

Por uma questão de simplicidade, suponha que tenhamos um pequeno vocabulário de apenas 6 palavras (em vez das 50.257 palavras no vocabulário do tokenizador BPE) e queiramos criar embeddings de tamanho 3 (no GPT-3, o tamanho do embedding é de 12.288 dimensões):

```
Python  
vocab_size = 6  
output_dim = 3
```

Usando o `vocab_size` e `output_dim`, podemos instanciar uma camada de embedding no PyTorch, definindo a semente aleatória como 123 para fins de reprodutibilidade:

```
Python
torch.manual_seed(123)
embedding_layer = torch.nn.Embedding(vocab_size, output_dim)
print(embedding_layer.weight)
```

A instrução `print` exibe a matriz de pesos subjacente da camada de embedding:

```
Python
Parameter containing:
tensor([[ 0.3374, -0.1778, -0.1690],
        [ 0.9178,  1.5810,  1.3010],
        [ 1.2753,  0.2010, -0.1606],
        [-0.4015,  0.9666, -1.1481],
        [-1.1589,  0.3255, -0.6315],
        [-2.8400, -0.7849, -1.4096]], requires_grad=True)
```

A matriz de pesos da camada de embedding contém valores pequenos e aleatórios. Esses valores são otimizados durante o treinamento do LLM como parte da própria otimização do LLM. Além disso, podemos ver que a matriz de pesos tem seis linhas e três colunas. Há uma linha para cada um dos seis tokens possíveis no vocabulário e uma coluna para cada uma das três dimensões de embedding.

Agora, vamos aplicá-la a um ID de token para obter o vetor de embedding:

```
Python
print(embedding_layer(torch.tensor([3])))
```

O vetor de embedding retornado é:

```
Python
tensor([-0.4015,  0.9666, -1.1481]), grad_fn=<EmbeddingBackward0>)
```

Se compararmos o vetor de embedding para o ID de token 3 com a matriz de embedding anterior, vemos que ele é idêntico à quarta linha (o Python começa com índice zero, então é a linha correspondente ao índice 3). Em outras palavras, a camada de embedding é essencialmente uma operação de busca (*lookup*) que recupera linhas da matriz de pesos da camada de embedding por meio de um ID de token.

NOTA: Para aqueles que estão familiarizados com codificação one-hot (*one-hot encoding*), a abordagem da camada de embedding descrita aqui é essencialmente apenas uma maneira mais eficiente de implementar a codificação one-hot seguida por multiplicação de matrizes em uma camada totalmente conectada, o que é ilustrado no código suplementar no GitHub em [\[https://mng.bz/ZEB5\]](https://mng.bz/ZEB5)(<https://mng.bz/ZEB5>). Como a camada de embedding é apenas uma implementação mais eficiente equivalente à abordagem de

codificação one-hot e multiplicação de matrizes, ela pode ser vista como uma camada de rede neural que pode ser otimizada via retropropagação.

Vimos como converter um único ID de token em um vetor de embedding tridimensional. Vamos agora aplicar isso a todos os quatro IDs de entrada (`torch.tensor([2, 3, 5, 1])`):

```
Python
print(embedding_layer(input_ids))
```

A saída do `print` revela que isso resulta em uma matriz 4×3 :

```
Python
tensor([[ 1.2753,  0.2010, -0.1606],
        [-0.4015,  0.9666, -1.1481],
        [-2.8400, -0.7849, -1.4096],
        [ 0.9178,  1.5810,  1.3010]], grad_fn=<EmbeddingBackward0>)
```

Cada linha nesta matriz de saída é obtida por meio de uma operação de busca a partir da matriz de pesos de embedding, conforme ilustrado na Figura 2.16.

Figura 2.16

As camadas de embedding realizam uma operação de busca, recuperando o vetor de embedding correspondente ao ID de token a partir da matriz de pesos da camada de embedding. Por exemplo, o vetor de embedding do ID de token 5 é a sexta linha da matriz de pesos da camada de embedding (é a sexta em vez da quinta linha porque o Python começa a contar em 0). Assumimos que os IDs de token foram produzidos pelo pequeno vocabulário da seção 2.3.

Tendo agora criado vetores de embedding a partir de IDs de token, em seguida adicionaremos uma pequena modificação a esses vetores de embedding para codificar informações posicionais sobre um token dentro de um texto.

2.8 Codificando posições de palavras

Em princípio, embeddings de token são uma entrada adequada para um LLM. No entanto, uma pequena limitação dos LLMs é que seu mecanismo de autoatenção (*self-attention*, veja o Capítulo 3) não possui uma noção de posição ou ordem para os tokens dentro de uma sequência. A forma como a camada de embedding apresentada anteriormente funciona é que o mesmo ID de token sempre é mapeado para a mesma representação vetorial, independentemente de onde o ID de token esteja posicionado na sequência de entrada, como mostrado na Figura 2.17.

Figura 2.17

A camada de embedding converte um ID de token na mesma representação vetorial, independentemente de onde ele esteja localizado na sequência de entrada. Por exemplo, o ID de token 5, esteja ele na primeira ou na quarta posição no vetor de entrada de IDs de token, resultará no mesmo vetor de embedding.

Em princípio, a incorporação determinística e independente de posição do ID de token é boa para fins de reprodutibilidade. No entanto, como o mecanismo de autoatenção dos próprios LLMs também é agnóstico em relação à posição, é útil injetar informações de posição adicionais no LLM.

Para conseguir isso, podemos usar duas amplas categorias de embeddings conscientes da posição: **embeddings posicionais relativos** e **embeddings posicionais absolutos**. Embeddings posicionais absolutos estão diretamente associados a posições específicas em uma sequência. Para cada posição na sequência de entrada, um embedding único é adicionado ao embedding do token para transmitir sua localização exata. Por exemplo, o primeiro token terá um embedding posicional específico, o segundo token outro embedding distinto, e assim por diante, conforme ilustrado na Figura 2.18.

Figura 2.18

Embeddings posicionais são adicionados ao vetor de embedding de token para criar os embeddings de entrada para um LLM. Os vetores posicionais têm a mesma dimensão dos embeddings de token originais. Os embeddings de token são mostrados com valor 1 por simplicidade.

- **Embeddings de entrada:** [2.1, 2.2, 2.3] | [3.1, 3.2, 3.3] | [4.1, 4.2, 4.3] | [5.1, 5.2, 5.3]
- **Embeddings posicionais:** [1.1, 1.2, 1.3] | [2.1, 2.2, 2.3] | [3.1, 3.2, 3.3] | [4.1, 4.2, 4.3]
- **Embeddings de token:** [1, 1, 1] | [1, 1, 1] | [1, 1, 1] | [1, 1, 1]

Em vez de focar na posição absoluta de um token, a ênfase dos embeddings posicionais relativos está na posição relativa ou distância entre os tokens. Isso significa que o modelo aprende as relações em termos de "quão distantes" em vez de "em qual posição exata". A vantagem aqui é que o modelo pode generalizar melhor para sequências de comprimentos variáveis, mesmo que não tenha visto tais comprimentos durante o treinamento.

Ambos os tipos de embeddings posicionais visam aumentar a capacidade dos LLMs de entender a ordem e as relações entre os tokens, garantindo previsões mais precisas e conscientes do contexto. A escolha entre eles geralmente depende da aplicação específica e da natureza dos dados que estão sendo processados.

Os modelos GPT da OpenAI usam **embeddings posicionais absolutos** que são otimizados durante o processo de treinamento em vez de serem fixos ou pré-definidos como as codificações posicionais no modelo transformer original. Esse processo de otimização faz parte do treinamento do próprio LLM.